

Technical Architecture Document

AssetFlow AI

System Architecture & Backend Design

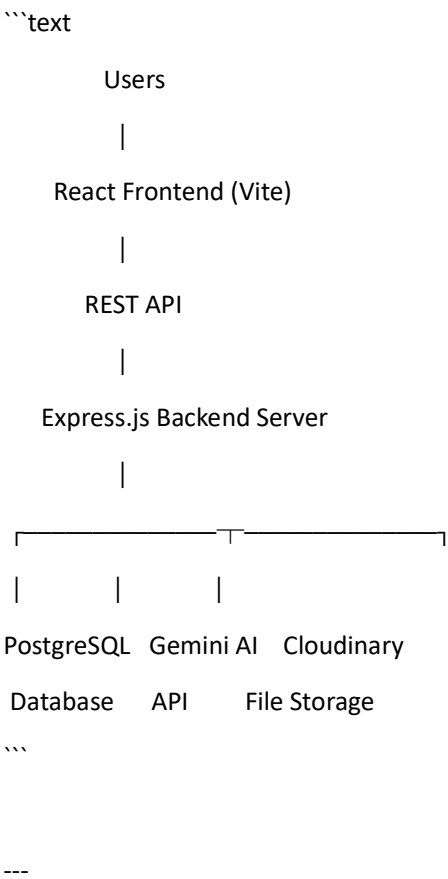
Version: 1.0

Table of Contents

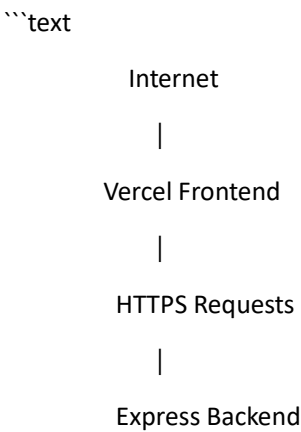
1. System Overview
2. Architecture Diagram
3. Technology Stack
4. High-Level Architecture
5. Folder Structure
6. Backend Architecture
7. Frontend Architecture
8. Database Architecture
9. API Layer
10. Authentication Flow
11. AI Services
12. File Storage
13. Notification System
14. Logging & Monitoring
15. Deployment
16. Scalability
17. Future Enhancements

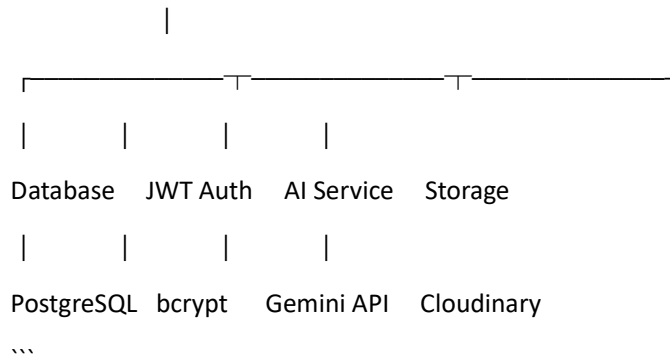
1. System Overview

AssetFlow AI follows a modern 3-tier architecture.



2. Architecture Diagram





3. Technology Stack

Frontend

- React 19
- TypeScript
- Vite
- Tailwind CSS
- React Router
- Redux Toolkit
- Axios
- React Hook Form
- Zod
- Framer Motion
- Recharts
- TanStack Table

Backend

- Node.js
- Express.js
- TypeScript
- Prisma ORM
- JWT
- bcrypt
- Multer
- Nodemailer

Database

PostgreSQL

AI

Google Gemini API

OCR

Tesseract.js

Storage

Cloudinary

Deployment

Frontend

Vercel

Backend

Render

Database

Supabase PostgreSQL

4. High-Level Architecture

```text

Frontend

- |
- ├─ Authentication
- ├─ Dashboard
- ├─ Assets
- ├─ Maintenance
- ├─ Reports
- └─ AI Assistant

↓

REST API

↓

Backend

↓

Business Logic

↓

Database

...

---

## # 5. Folder Structure

```
```text
```

```
assetflow-ai/
```

```
├── frontend/
```

```
|
```

```
| ├── public/
```

```
| ├── src/
```

```
| |
```

```
| ├── assets/
```

- | └─ components/
- | └─ layouts/
- | └─ pages/
- | └─ hooks/
- | └─ services/
- | └─ context/
- | └─ redux/
- | └─ utils/
- | └─ routes/
- | └─ types/
- | └─ main.tsx
- |
- | └─ backend/
- |
- | └─ prisma/
- | └─ src/
- | |
- | └─ config/
- | └─ controllers/
- | └─ middleware/
- | └─ models/
- | └─ routes/
- | └─ services/
- | └─ repositories/
- | └─ utils/
- | └─ validators/
- | └─ ai/
- | └─ jobs/
- | └─ server.ts
- |

```
├── docs/
├── README.md
└── docker-compose.yml
...
```

6. Backend Architecture

```
```text
Client Request
 |
 ▼
Routes
 |
 ▼
Middleware
 |
 ▼
Controller
 |
 ▼
Service
 |
 ▼
Repository
 |
 ▼
Database
...
```
```


Responsibilities

Routes

- URL Mapping

Controllers

- Receive Request
- Validate Input

Services

- Business Logic

Repositories

- Database Queries

Middleware

- Authentication
- Authorization
- Logging

7. Frontend Architecture

```text

Pages

- |
- └─ Dashboard
- └─ Assets
- └─ Maintenance
- └─ Reports
- └─ AI Assistant
- └─ Settings



Components



API Services



Backend

...

---

# 8. Database Architecture

Core Tables

```text

Users

Departments

Assets

Assignments

Maintenance

Vendors

Invoices

Notifications

AssetHistory

AuditLogs

...

Relationship

```text

Department

↓

Users

↓

Assignments

↓

Assets

↓

Maintenance

↓

## History

...

---

## # 9. API Layer

### Authentication

...

POST /api/auth/login

POST /api/auth/register

POST /api/auth/logout

GET /api/auth/me

...

### Assets

...

GET /api/assets

POST /api/assets

GET /api/assets/:id

PUT /api/assets/:id

DELETE /api/assets/:id

...

### Assignments

...

POST /api/assignments

PUT /api/assignments/:id

GET /api/assignments

...

## Maintenance

...

POST /api/maintenance

GET /api/maintenance

PUT /api/maintenance/:id

...

## Dashboard

...

GET /api/dashboard

...

## Reports

...

GET /api/reports

...

## AI

...

POST /api/ai/chat

POST /api/ai/predict

POST /api/ai/ocr

...

---

## # 10. Authentication Flow

```text

Login



Validate Credentials



bcrypt Compare



Generate JWT



Return Token



Protected Routes



Middleware Validation



Access Granted

...

11. AI Services

AI Chat

Purpose

Natural language queries

Example

Show laptops older than 4 years.

Predictive Maintenance

Input

- Purchase Date
- Repair History
- Usage Hours

Output

Risk Score

Example

87%

Maintenance Required

YES

OCR Service

Input

Invoice Image

↓

Extract

Vendor

Invoice Number

Purchase Date

Warranty

Amount



Auto-fill Form

12. File Storage

Uploads

Invoices

Images

Repair Documents

Storage

Cloudinary

Returned URL stored in database

13. Notification System

Channels

Email

In-App

Future

SMS

Push Notification

Triggers

Maintenance Due

Warranty Expiring

Assignment Completed

Asset Returned

Low Inventory

14. Logging & Monitoring

Application Logs

Morgan

Error Logs

Winston

Audit Logs

Database Table

Health Check

...

GET /health

...

Returns

...

```
{  
  "status": "UP"  
}
```

...

15. Deployment

Frontend

Vercel

Backend

Render

Database

Supabase PostgreSQL

Storage

Cloudinary

Environment Variables

...

DATABASE_URL=

JWT_SECRET=

GEMINI_API_KEY=

CLOUDINARY_NAME=

CLOUDINARY_KEY=

CLOUDINARY_SECRET=

...

16. Scalability

Future Improvements

Redis Cache

CDN

Microservices

Message Queue

Docker

Kubernetes

Auto Scaling

Load Balancer

17. Future Enhancements

RFID Tracking

IoT Sensors

GPS Tracking

Offline Sync

Mobile App

Voice Assistant

Blockchain Asset Ledger

ERP Integration

Predictive Budget Planning

Digital Twin

System Principles

✓ Clean Architecture

✓ SOLID Principles

✓ Modular Codebase

✓ Secure APIs

✓ Scalable Infrastructure

✓ AI-First Design

✓ Cloud Native

✓ Easy to Maintain

✓ Ready for Production